

Pi-CloudDOOM — Architecture Overview

A single-page bird's-eye view of the whole platform: services, data, security, observability, and how they all fit together.

For deeper dives: - Deployment strategy → `DEPLOYMENT_REPORT.md` - CI/CD pipeline → `CICD_REPORT.md` - Live demo script → `JURY_DEMO_GUIDE.md` - Per-step build history → `docs/cicd/step-00..08-*.md`

1. What is Pi-CloudDOOM?

InterviewPrep TN — a microservice platform for interview preparation: - Take mock interviews (voice + avatar, real-time). - Get an AI-generated personalized training path. - Earn XP, badges, complete quizzes. - Be matched with mentors. - Read learning resources, discuss in a community feed.

The project is the deployment thesis: a polyglot, multi-tenant, real-world workload running across three clouds.

2. Service inventory

Service	Stack	Port	Plane	Purpose
<code>user-service</code>	Spring Boot 4.x / Java 21	8081	OpenStack K8s	Users, profiles, RBAC, CV upload + parsing (Groq), passkeys
<code>interview-service</code>	Spring Boot 4.x / Java 21	8082	OpenStack K8s	Mock interview sessions, live voice, avatar (Simli + Kokoro TTS)
<code>training-service</code>	Spring Boot 4.x / Java 21	8083	OpenStack K8s	Training paths, gamification, XP, training coach

Service	Stack	Port	Plane	Purpose
mentorship-service	Spring Boot 4.x / Java 21	8084	OpenStack K8s	Mentor matching, sessions, ratings, email notifications
quiz-service	Spring Boot 3.4.1 / Java 17	8085	OpenStack K8s	Quizzes + oral attempts
community-service	Spring Boot 4.x / Java 21	8086	OpenStack K8s	Community feed
resource-service	Spring Boot 4.x / Java 21	8087	OpenStack K8s	Learning resources catalogue
ai-training-path	FastAPI / Python 3.12	8001	Azure ACA	ML/LLM- driven training path generation
kokoro	FastAPI / Python	—	Azure ACA	Text-to- speech for the interview avatar
ollama	Ollama runtime + llama3.2:3b	—	Azure ACA	Local LLM serving
whisper (3rd-party)	fedirz/faster-whisper-server	—	Azure ACA	Speech-to- text
frontend	Angular 21	4200 (dev)	Vercel	Web UI
attack-detector	Python / Prometheus exporter	8000	OpenStack K8s	AI-based attack detection (sidecar component)

Stateful infrastructure (OpenStack K8s): - PostgreSQL 16 (multi-database, single instance) — backs every Spring service + Keycloak. - Redis 7 — session/cache. - Kafka 7.5 (KRaft, no ZooKeeper) — event bus, 14 topics. - MinIO (latest) — S3-compatible object storage. - Keycloak 24 — OIDC provider, realm myapp-realm imported from JSON.

3. Logical architecture

user (browser)	Cloudflare (edge)	TLS termination + DNS
----------------	----------------------	-----------------------

Vercel CDN Angular SPA	Cloudflare Tunnel (cloudflared pod)	Azure CA fqdns ai-training- path / kokoro /
interviewprep-tn.me	ollama	/ whisper

Traefik (k8s) ingress-node hostPort 80/443 CORS middlw.	HTTPS direct from Spring services
--	---

api ingress.yaml (path-based routing to 7 services)	auth ingress.yaml → Keycloak
---	------------------------------------

user-svc 8081	intvw 8082	train 8083	mentor 8084	quiz 8085	comm 8086	rsrc 8087
------------------	---------------	---------------	----------------	--------------	--------------	--------------

Postgres 16 PVC 10Gi	Redis 7	Kafka KRaft PVC 5Gi	MinIO S3 API PVC 5Gi
14 topics			

4. Data flows

4.1 User logs in

1. Browser → Vercel (Angular SPA).
2. Angular hits `auth.interviewprep-tn.me` (Cloudflare → Traefik → Keycloak).
3. Keycloak OIDC dance → returns access + refresh tokens.
4. Subsequent API calls go to `api.interviewprep-tn.me` with `Authorization: Bearer ....`
5. Each Spring service validates the JWT against Keycloak's JWK URI (cached, configurable in `app-config`).

4.2 Mock interview (voice + avatar)

1. Frontend opens a session → `interview-service` (path `/api/live-interviews`).
2. `interview-service` calls `whisper` (Azure ACA) to transcribe the audio chunks.
3. `interview-service` calls `ollama` (Azure ACA) for `llama3.2:3b` responses.
4. `interview-service` calls `kokoro` (Azure ACA) for TTS audio.
5. Avatar lip-sync handled client-side by Simli (3rd-party SaaS).
6. Final score event published to Kafka topic `interview.session.completed` → `training-service` updates XP.

4.3 Personalized training path

1. Frontend → `training-service` `/api/v1/training/personalize`.
2. `training-service` calls `ai-training-path` (Azure ACA) at `/predict-path` — sends user profile vector.
3. `ai-training-path` returns the ranked path.
4. `training-service` persists it in Postgres, publishes `training.path.created` to Kafka.

4.4 CV parsing

1. User uploads CV → `user-service` `/api/users/cv` → stored on the `user-uploads-pvc`.
2. `user-service` extracts text (PDFBox).
3. `user-service` calls **Groq** API (external, not deployed by us) with `llama-3.3-70b-versatile` for structured extraction (was previously Ollama; switched in commit `372ed81c` for latency).
4. Parsed result returned to frontend.

5. Security model

Layer	Mechanism
Edge TLS	Cloudflare termination, free auto-renewed cert
DDoS / WAF	Cloudflare edge (free tier)
Inbound to cluster	Cloudflare Tunnel (cloudflared pod) — no public port on OpenStack
Service-to-service auth	Keycloak OIDC JWTs validated per request
DB credentials	K8s Secrets only — never inlined in manifests; app-secrets Secret
CI/CD blast radius	Namespace-scoped cicd-deployer SA — no cluster-admin, no Secret reads, no RBAC writes
Image provenance	Cosign keyless OIDC signing on every push; SBOM + SLSA provenance attached
Image scanning	Trivy in build-and-push.yml (SARIF to GitHub Code Scanning)
Static analysis (code)	CodeQL security-extended , security-and-quality queries on Java, JS/TS, Python, Actions
Static analysis (IaC)	Trivy config + kube-linter on k8s/** + infra/** (report-only day-1)
Secret scanning	Gitleaks on PR + push + weekly full-history sweep
Dependency hygiene	Dependabot weekly grouped PRs across all ecosystems
Runtime detection	AI attack detector (attack-detection namespace) → Prometheus alerts
RBAC	Namespace-scoped roles; principle of least privilege

6. Observability

6.1 Logs

- Spring services log to stdout in JSON; viewable via `kubectl logs`.
- The `deploy-k8s.yml` workflow auto-dumps `kubectl describe` + last 80 log lines per pod when a rollout fails.

- Frontend uses `console` + Sentry-style hooks (not configured today).

6.2 Metrics

kube-prometheus-stack installed via Helm (`k8s/monitoring/monitoring-values.yaml`):

- Prometheus (7-day retention, `serviceMonitorSelector: {}` so it scrapes the whole cluster).
- Alertmanager (ClusterIP, default routing).
- Grafana (ClusterIP, admin password seeded via Helm values).
- kube-state-metrics + node-exporter.

Spring services expose `/actuator/prometheus` (when enabled in `application.yml`).

6.3 Alerts

Defined in PrometheusRule CRDs. Two custom rules in `k8s/attack-detection/prometheus-rules.yaml`:

- `AIThreatDetected` — `max_over_time(attack_label[1m]) >= 1` for 15 s.
- `AttackDetectorDown` — `absent(attack_detector_up)` for 2 m.

6.4 Dashboards

Two committed JSON dashboards in `k8s/attack-detection/`: - `attack-detector-dashboard.json`

- model-level metrics (label distribution, version, attack rate).
- `ai-attack-detector-k8s-dashboard.json`
- K8s-side metrics on the detector pod.

6.5 Pipeline observability

- Every workflow writes a `$GITHUB_STEP_SUMMARY`: - Build summary → image, digest, tags.
- Deploy summary → image pinned, rollout exit code, pod listing.
- Vercel prod → deployment URL + commit SHA.

So a developer opening a workflow run sees the whole story without reading 800 log lines.

7. Branching strategy

Branch	Purpose
<code>main</code>	Released code, protected. Triggers prod deploys.
<code>deployment/hybrid-openstack-azure-Carcelit</code>	Current active deployment branch. Also auto-deploys (via <code>workflow_run</code>).
<code>integration/*</code>	Integration branches per feature team (m2 = interview, m4 = training, m5 = mentorship, m6 = community, ...)

Branch	Purpose
feature/*	Per-feature work — PR back into integration
dependabot/*	Auto-generated weekly grouped dep PRs
copilot/*	GitHub Copilot Workspace branches (analyze repo, identify perf issues)

8. The 13 GitHub Actions workflows (at a glance)

CI scaffolding (3 reusable + 1 orchestrator):

```
ci.yml      ci-spring-service.yml      (×7 services)
             ci-python-service.yml      (ai-training-path)
             ci-frontend.yml            (Angular)
```

Image plane (1):

```
build-and-push.yml      (matrix × 10 → DockerHub, signed)
```

Deploy plane (3):

```
deploy-k8s.yml          (matrix × 7 → picloudboom / picloudboom-dev)
deploy-aca.yml          (matrix × 3 → Azure Container Apps)
vercel-preview.yml + vercel-prod.yml (Angular → Vercel)
```

Security plane (3):

```
codeql.yml      (×4 languages)
iac-scan.yml     (Trivy + kube-linter on k8s/ + infra/)
gitleaks.yml     (full git history)
```

Maintenance (1):

```
dependabot.yml (weekly grouped PRs)
```

9. Cloud bill of materials

Cloud	What runs here	Cost model
OpenStack (school infra)	Heat cluster: 5 nodes, K8s 1.29.15, Postgres, Redis, Kafka, MinIO, Keycloak, 7 Spring services, attack-detector, Prometheus stack	Free (in-house)

Cloud	What runs here	Cost model
Azure (rg-picclouddoom, France Central)	4 Container Apps: ai-training-path, kokoro, ollama, whisper	Pay-per-second active CPU + provisioned memory; scale-to-zero
Vercel (team_tQYcRewNfvSBGAbsE6wJwcp)	Angular frontend	Free hobby tier
DockerHub	10 public image repos under azizbna/pi-clouddoom-*	Free
Cloudflare	DNS + Tunnel + edge TLS	Free
GitHub	Repo + Actions + Code Scanning + Dependabot	Free (public repo)
External APIs	Groq (LLM), Google AI Studio (Gemini for training coach), Mailtrap sandbox	Free tier each

10. Repository map (top-level)

```

.
.github/
  workflows/                                # 13 GitHub Actions YAMLs
  dependabot.yml
.vercel/                                    # Vercel project link (org + project ID)
docs/
  cicd/                                     # 9 step-by-step pipeline docs + README
  DEPLOYMENT_REPORT.md                     # ← this set of reports
  CICD_REPORT.md
  JURY_DEMO_GUIDE.md
  ARCHITECTURE_OVERVIEW.md
infra/
  docker-compose.yml                       # local dev stack (all services)
  .env.example                             # required API keys for local dev
  openstack/
    heat-cluster.yaml                      # OpenStack IaC - the cluster as code
  keycloak/
    realm-export.json                     # Keycloak realm import
  db/init-multiple-databases.sh
  scripts/                                 # seed-test-users.sh, m4_smoke.sh
k8s/
  base/                                    # namespace + quota + ConfigMap
  infra/                                   # Postgres, Redis, Kafka, MinIO, Keycloak

```


apps/	# 7 Spring service manifests
ingress/	# Traefik values, Cloudflared, ingresses, CORS
monitoring/	# kube-prometheus-stack Helm values
attack-detection/	# AI security component + Grafana dashboards
rbac/	# cicd-deployer SA/Role/RoleBinding (prod + dev)
scripts/	
smoke.sh	# 2-mode smoke test (HTTP / kubectl)
extract-kubeconfig.sh	# mint SA-scoped kubeconfig for GH secret
deploy-aca.sh	# manual ACA deploy (laptop fallback)
frontend/	# Angular 21
user-service/	# Spring Boot
interview-service/	
training-service/	
mentorship-service/	
quiz-service/	
community-service/	
resource-service/	
ai-training-path/	# FastAPI
kokoro/	# TTS
ollama-deploy/	# baked LLM image
*.postman_collection.json	# API collections
vercel.json	# Vercel build config
README.md	

11. Future work (post-launch backlog)

1. **High availability** — multi-replica + PostgreSQL HA + Kafka 3-broker.
 2. **GitOps** with Argo CD — replace imperative `kubectl set image`.
 3. **Pre-deploy DB migrations** — separate Flyway runner job.
 4. **Smoke-test extension** — cover Azure Container Apps URLs + Vercel.
 5. **Federated OIDC for Azure** — replace service-principal secret.
 6. **Flip security gates from advisory to enforcing** — iac-scan and image-Trivy.
 7. **Kustomize overlays** for picloudboom-dev so dev is auto-populated.
 8. **Pod disruption budgets** + horizontal pod autoscalers.
 9. **OpenTelemetry traces** across the Spring services.
 10. **Multi-replica AI services** with custom KEDA scalers on ACA.
-

12. TL;DR

A 9-service polyglot platform, deployed across 3 clouds, automated by 13 GitHub Actions workflows. Every image is signed, every deploy is reversible, every secret stays in GitHub or K8s — never in source. The pipeline reads

top-to-bottom: CI → build → sign → deploy → smoke → rollback-on-fail.
Three lines of code change away from any failure mode being enforced instead of advisory.